

Practical Tasks

Task-1

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution

```
b.deepak@BDeepaks-MacBook-Air ~ % nano process.c
b.deepak@BDeepaks-MacBook-Air ~ %
```

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());

        sleep(5);

        printf("Child process terminated\n");
        exit(0);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for child...\n");
        wait(NULL);

        printf("Child completed. Parent process terminated.\n");
    }

    return 0;
}
```

```
b.deepak@BDeepaks-MacBook-Air ~ % nano process.c
b.deepak@BDeepaks-MacBook-Air ~ % gcc process.c -o process
b.deepak@BDeepaks-MacBook-Air ~ % ./process
```

Output:

```
b.deepak@BDeepaks-MacBook-Air ~ % gcc process.c -o process
b.deepak@BDeepaks-MacBook-Air ~ % ./process
Parent process started
Parent PID: 9158

Parent Process
Parent PID: 9158
Child PID: 9159
Parent is waiting for child...

Child Process
Child PID: 9159
Child PPID: 9158
Child process terminated
Child completed. Parent process terminated.
```

Task-2

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
b.deepak@BDeepaks-MacBook-Air ~ % nano process1.c
b.deepak@BDeepaks-MacBook-Air ~ % █
```

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <time.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process Started\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        printf("\nState 1: WAITING/SLEEPING\n");
        sleep(10);

        printf("\nState 2: RUNNING\n");

        time_t start = time(NULL);

        while (time(NULL) - start < 15) {
            volatile unsigned long long x = 0;
            for (unsigned long long i = 0; i < 10000000; i++) {
                x += i;
            }
        }

        printf("\nState 3: TERMINATING\n");
        exit(0);
    }
    else {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("\nParent is waiting for 35 seconds.\n");
        sleep(35);

        printf("\nParent calling wait().\n");
        wait(NULL);

        printf("Child process collected.\n");
        printf("Parent terminating.\n");
    }

    return 0;
}
```

```
[b.deepak@BDeepaks-MacBook-Air ~ % nano process1.c
[b.deepak@BDeepaks-MacBook-Air ~ % gcc process1.c -o process1
[b.deepak@BDeepaks-MacBook-Air ~ % ./process1
```

Output:

```
Parent Process
PID      : 9177
Child PID : 9178

Parent is waiting for 35 seconds.
Child Process Started
PID  : 9178
PPID : 9177

State 1: WAITING/SLEEPING

State 2: RUNNING

State 3: TERMINATING

Parent calling wait().
Child process collected.
Parent terminating.
```